

Solution 1

Solution 1 (a)

Online Algorithm Implementation: Mean

```

1  # load libraries
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import seaborn as sns
5
6  # quick check data-1.txt has 5000 pts
7  # cat data-1.txt | wc -l -> 5000
8
9  # load data-1.txt
10 data = np.loadtxt('data-1.txt')
11
12 def online_algorithm_mean(data):
13     mean_i = []
14     dt = []
15     current_mean = 0
16     for t,x_t in enumerate(data):
17         dt.append(t+1)
18         current_mean = current_mean + ((x_t-current_mean)/(t+1))
19         mean_i.append(current_mean)
20     print(f"final mean: {mean_i[-1]}")
21     return dt, mean_i
22
23 def plot_mean_dt(dt,mean_i,true_mean):
24     fig,ax = plt.subplots(nrows=1,ncols=1,figsize=(8,10))
25     sns.lineplot(x=dt,y=mean_i,markers='o',ax=ax,color='blue',label='OA')
26     sns.lineplot(x=dt,y=[true_mean for i in
27         dt],markers='--',ax=ax,color='red',label='True Mean')
28     ax.set_xlabel('dt')
29     ax.set_ylabel('mean')
30     ax.set_ylim(min(mean_i)-1e5,true_mean+1e5)
31     print(min(mean_i))
32     title = f"Online Algorithm(OA) for Mean\n True Mean: {true_mean:.3f} \n OA Mean:
33         {mean_i[-1]:.3f}"
34     ax.set_title(title)
35     fig.savefig('hw3_q1.png')
36
37 # calc true mean and print it
38 true_mean = sum(data)/len(data)
39 print(true_mean)
40
41 dt, mean_i = online_algorithm_mean(data)
42 plot_mean_dt(dt,mean_i,true_mean)

```

Comments

The true mean is 2,991,834.673 and is identical to the final mean calculated via `online_algorithm_mean(data)` function.

Solution 1

Solution 1 (b)

Plot

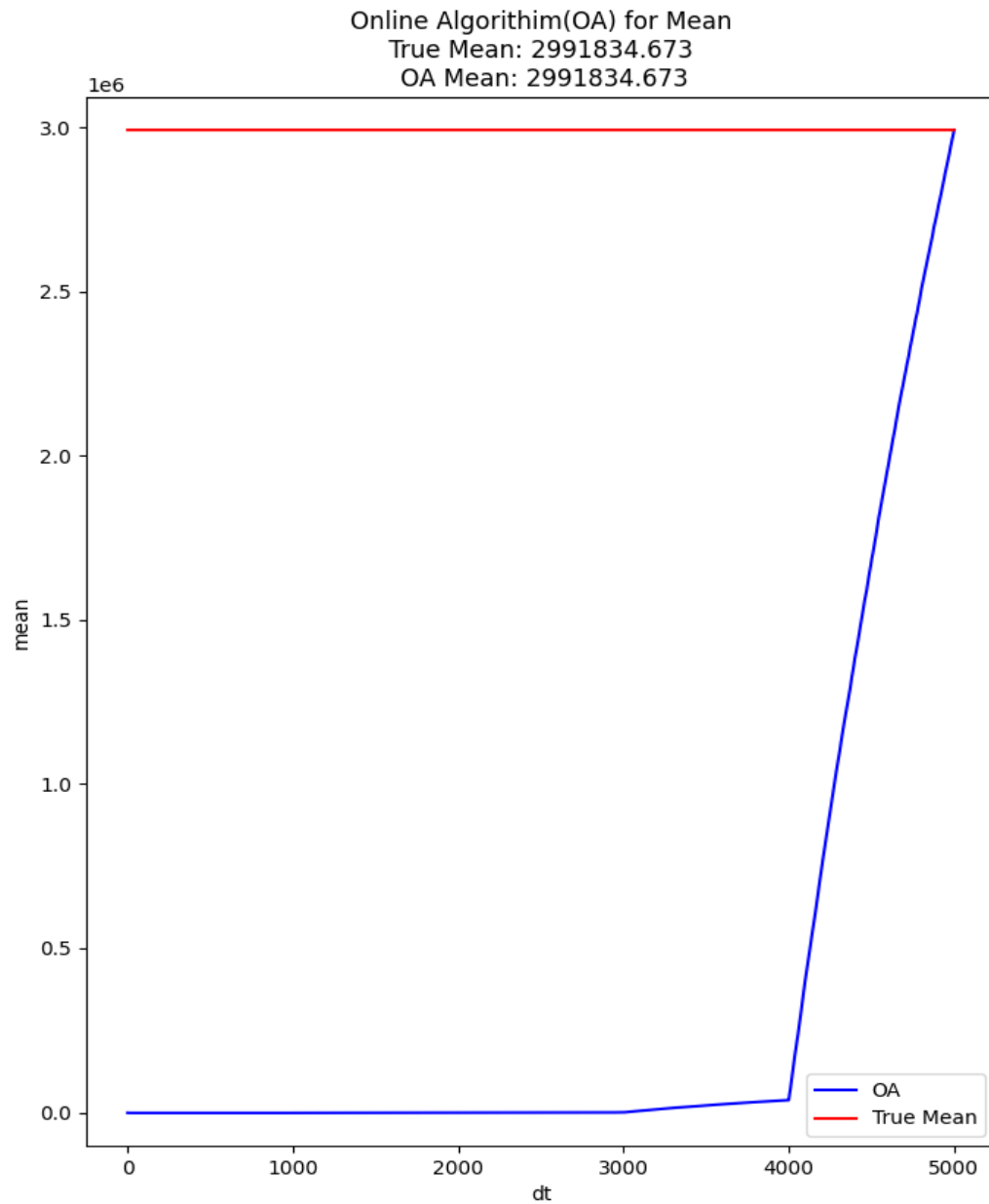


Figure 1: Figure shows all 5000 intermediate values of mean

Solution 2

Solution 2 (a)**Pseudocode**

Algorithm 1 Online Variance Calculation

Initialize:

$t \leftarrow 0$	▷ Count of data points
$\mu \leftarrow 0.0$	▷ Running mean of X
$M_2 \leftarrow 0.0$	▷ Running mean of X^2
$v \leftarrow 0.0$	▷ Running variance

For each new data point x_t :

$t \leftarrow t + 1$	
$\mu \leftarrow \mu + (x_t - \mu)/t$	▷ Update mean of X
$M_2 \leftarrow M_2 + (x_t^2 - M_2)/t$	▷ Update mean of X^2
$v \leftarrow M_2 - \mu^2$	▷ Calculate current variance

Solution 2

Solution 2 (b)

Online Algorithm Implementation: Variance

```
1  # load libraries
2  import numpy as np
3
4  # load data-1.txt
5  data = np.loadtxt('data-1.txt')
6
7  def online_algorithm_variance(data):
8      mean_x = 0.0
9      mean_x_squared = 0.0
10     variances = []
11
12     for t,x_t in enumerate(data):
13         # Update the mean of x
14         mean_x = mean_x + ((x_t-mean_x)/(t+1))
15         # Update the mean of x-squared
16         mean_x_squared = mean_x_squared + ((x_t**2-mean_x_squared)/(t+1))
17
18         variances.append(mean_x_squared - mean_x**2)
19
20     print(f"final variance: {variances[-1]:.3f}")
21     return variances
22
23 # calc true variance and print
24 true_variance = np.var(data)
25 print(f"true variance: {true_variance:.3f}")
26
27 # calc online_algorithm_variance
28 variances = online_algorithm_variance(data)
```

Comments

The true variance using numpy: 36,618,966,207,041.938 while the variance calculated via `online_algorithm_variance(data)` was: 36,618,966,207,041.961. This discrepancy between the true and *online variance calculation* arises because the online formula is sensitive to floating point precision errors, especially when subtracting two large, similar numbers.

Solution 3

Solution 3 (a)

True Median

```
1 # load libraries
2 import numpy as np
3
4 # load data-1.txt
5 data = np.loadtxt('data-1.txt')
6
7 # calc true median
8 true_median = np.median(data)
9 print(f"true median: {true_median:.3f}")
```

The true median using numpy: 1,526

Solution 3

Solution 3 (b)/(c)

Random Sample with Replacement Algorithm Implementation: Median

```

1  # load libraries
2  import numpy as np
3  import random
4
5  # load data-1.txt
6  data = np.loadtxt('data-1.txt')
7
8  def estimate_online_median(data, k, repetitions):
9
10     print(f"--- Starting estimations for k={k} ---")
11     for rep in range(repetitions):
12         # initialize the sample array 's' with the first data point.
13         # from s1=s2=...=sk=x1
14         s = np.full(k, data[0])
15
16         # get x_t from t=2,3,...,t
17         for i in range(1, n):
18             t = i + 1
19             x_t = data[i]
20
21             # for j=1,2,...,k then sj = xt with probability 1/t.
22             for j in range(k):
23                 if random.random() < (1.0 / t):
24                     s[j] = x_t
25
26         # calculate the median of the final sample for each repetition
27         estimated_median = np.median(s)
28         print(f"repetition {rep + 1:2d}: estimated median = {estimated_median}")
29     print("-" * (35 + len(str(k))))
30
31 # part (a): calc and print the true median
32 true_median = np.median(data)
33 print("--- true median calculation ---")
34 print(f"true median: {true_median}\n")
35
36 # part (b): simulation for k=100
37 estimate_online_median(data, k=100, repetitions=10)
38
39 # part (c): simulation for k=500
40 estimate_online_median(data, k=500, repetitions=10)
41

```

Solution 3**Solution 3 (b)/(c)****Random Sample with Replacement Algorithm Results: Median**

(a) Estimated Medians for $k = 100$		(b) Estimated Medians for $k = 500$	
Repetition	Estimated Median	Repetition	Estimated Median
1	1750.0	1	1422.0
2	1094.0	2	1498.5
3	1236.0	3	1404.5
4	1166.0	4	1328.5
5	1552.5	5	1592.5
6	1338.5	6	1452.5
7	1583.5	7	1520.5
8	1635.0	8	1444.5
9	1805.5	9	1541.5
10	1376.0	10	1567.0

Table 1: Comparison of estimated medians for different sample sizes. *Note: as k grows sufficiently large the calculation for the median of a sample using the random sample with replacement should converge to the true median.*

Solution 4

An ϵ -heavy hitter is an element in a data stream of length m whose frequency is greater than or equal to ϵm . We need to find the maximum number of such elements when $\epsilon = 0.05$.

Let k be the number of distinct ϵ -heavy hitters. Let the frequency of the i -th heavy hitter be denoted by f_i .

By definition, for each of the k heavy hitters:

$$f_i \geq \epsilon m$$

The sum of frequencies of these distinct items cannot exceed the total stream length m :

$$\sum_{i=1}^k f_i \leq m$$

Substitute the minimum frequency for each heavy hitter into the sum:

$$\begin{aligned} \sum_{i=1}^k (\epsilon m) &\leq \sum_{i=1}^k f_i \leq m \\ k \cdot \epsilon \cdot m &\leq m \end{aligned}$$

Since $m > 0$, we can divide both sides by m :

$$k \cdot \epsilon \leq 1$$

Solving for k :

$$\begin{aligned} k &\leq \frac{1}{\epsilon} \\ k &\leq \frac{1}{0.05} \\ k &\leq 20 \end{aligned}$$

$\therefore k \leq 20$ demonstrates that for $\epsilon = 0.05$, there can be at most 20 ϵ -heavy hitters in the stream.

Solution 5

Solution 5 (a)

something something something

Solution 5

Solution 5 (b)

something something something

Solution 6

Solution 6 (a)

something something something

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 """
4 code different
5 """
6 data = np.loadtxt
```

Solution 6

Solution 6 (b)

something something something

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 """
4 code different
5 """
6 data = np.loadtxt
```

Solution 7

Solution 7 (a)

something something something

Solution 7

Solution 7 (b)

something something something
